

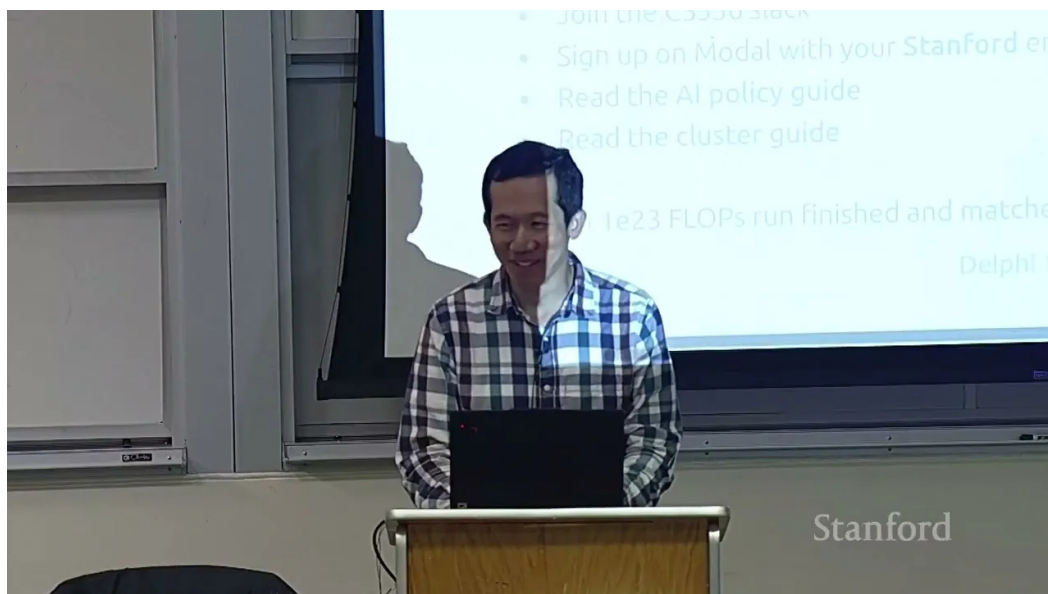
课程笔记

PyTorch、资源核算与算术强度

Stanford CS336 Spring 2026 · Lecture 2 高密度中文讲义

Percy Liang (讲授) / lecture-to-notes (整理)

2026 年 7 月 11 日



视频作者 / 频道: Stanford Online

发布日期: 2026-04-14

视频时长: 01:17:24

视频链接: <https://www.youtube.com/watch?v=kuYAsz7zspQ>

课程主页: <https://cs336.stanford.edu/>

官方可执行课件: `lecture_02.py`

目录

1 资源核算：先把训练问题变成数量级	3
1.1 70B 参数、15T token、1024 张 H100 需要多久	3
1.2 8 张 80GB H100 最多能装多少参数	4
1.3 本章小结	5
2 张量是资源核算的统一对象	5
2.1 秩、形状与语义维度	5
2.2 显存等于元素数乘元素宽度	5
2.3 本章小结	6
3 数值格式：容量、范围与分辨率的三角折中	6
3.1 FP16：省一半显存，但小数可能直接归零	7
3.2 BF16：保留 FP32 的范围，牺牲分辨率	7
3.3 混合精度：不同状态承担不同数值责任	8
3.4 FP8 与 NVFP4：位宽继续下降后必须引入尺度管理	9
3.5 本章小结	10
4 设备放置与具名张量运算	10
4.1 CPU 与 GPU 之间的移动也是成本	10
4.2 Einsum：把维度语义写进运算	11
4.3 Reduce 与 Rearrange：把形状变换变成类型检查	12
4.4 本章小结	13
5 FLOPs：从矩阵形状推导计算量	13
5.1 矩阵乘法为何是 2BDK	14
5.2 其他算子通常不是主导 FLOPs，却可能主导时间	15
5.3 本章小结	15
6 正确基准测试与 MFU	15
6.1 GPU 异步执行为何会让计时失真	15
6.2 MFU：理论峰值到底兑现了多少	16
6.3 本章小结	17
7 算术强度：计算与数据搬运谁更慢	17
7.1 ReLU：FLOPs 很少，时间却由带宽决定	18

7.2	工作负载强度与硬件临界强度	19
7.3	本章小结	21
8	Roofline: 矩阵乘法为何适合训练, 矩阵向量为何拖慢推理	21
8.1	几类操作的强度对比	21
8.2	训练与自回归推理的结构差异	22
8.3	本章小结	22
9	Autograd 与反向传播的计算账本	22
9.1	Autograd 记录什么	23
9.2	单层反向为何是前向的两倍	24
9.3	全网络的 6BP 近似	25
9.4	本章小结	26
10	训练显存: 静态状态与随 batch 增长的激活	26
10.1	生命周期决定峰值, 而不是简单总和	27
10.2	本章小结	27
11	用计算换显存: 梯度累积与激活检查点	28
11.1	梯度累积: 把一个大 batch 切成多个微批	28
11.2	激活检查点: 前向少存, 反向重算	28
11.3	检查点频率的复杂度折中	29
11.4	本章小结	30
12	总结与延伸	31
12.1	讲者总结	31
12.2	结构化提炼: 四张表贯穿后续课程	31
12.3	建议练习	32
12.4	拓展阅读	32
12.5	本章小结	32

1 资源核算：先把训练问题变成数量级

本讲的主题不是某个神奇的新模型，而是一种工程习惯：在开始实现之前，先把计算量、显存容量、数据搬运和墙钟时间写成可检查的数量级。固定资源下的目标是训练出最好的模型；要讨论“最好”，就必须先知道一个计算会消耗多少资源。

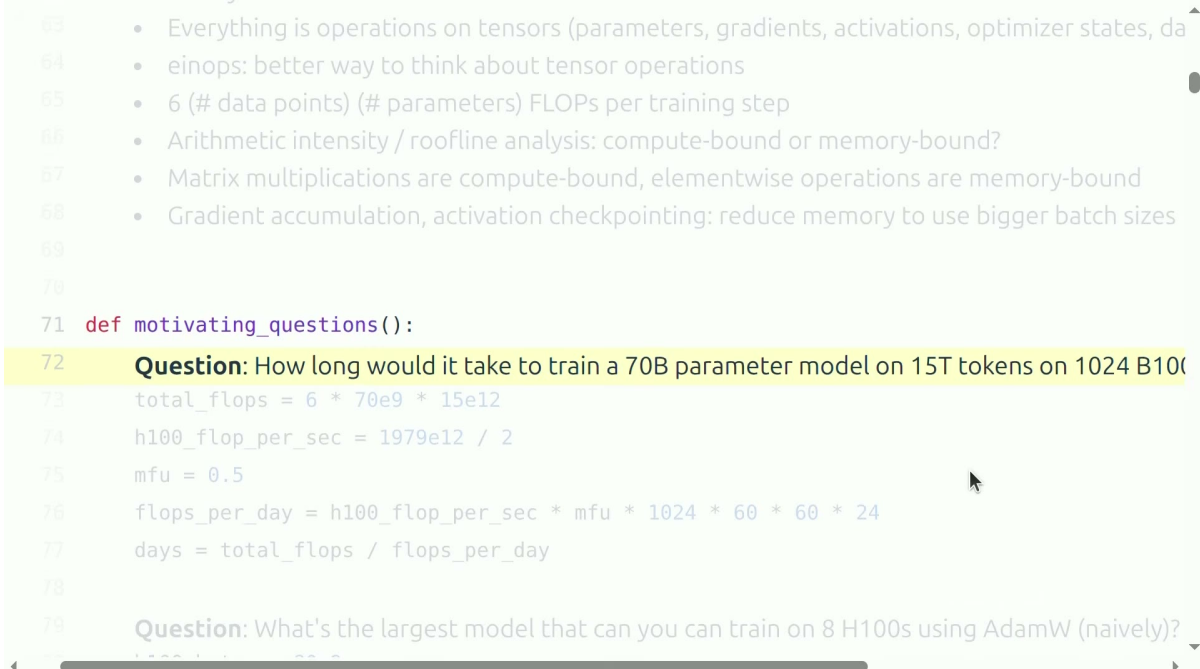


图 1: 本讲的出发点：给定固定计算与显存预算，先回答训练需要多久、模型能装多大。¹

1.1 70B 参数、15T token、1024 张 H100 需要多久

对以稠密矩阵乘法为主、上下文不太长的 Transformer，训练总计算量常用

$$C_{\text{train}} \approx 6PT$$

估算。若 $P = 70 \times 10^9$ 、 $T = 15 \times 10^{12}$ ，则

$$C_{\text{train}} \approx 6.3 \times 10^{24} \text{ FLOPs.}$$

其中：

- C_{train} ：完整训练所执行的浮点运算总数；
- P ：模型参数量；
- T ：训练中处理的 token 数；
- 系数 6：前向约 $2PT$ 、反向约 $4PT$ 的合计近似。

课件取 H100 稠密 BF16 峰值 $F_{\text{peak}} = 1979/2 \text{ TFLOP/s}$ 、模型 FLOPs 利用率 $u = 0.5$ 、GPU 数 $G = 1024$ ，于是每天的有效计算量为

$$C_{\text{day}} = F_{\text{peak}} uG (86400) \approx 4.377 \times 10^{22} \text{ FLOPs/day,}$$

¹视频画面时间区间：00:02:00–00:02:15。

训练时间约为

$$D_{\text{train}} = \frac{C_{\text{train}}}{C_{\text{day}}} \approx 143.9 \text{ days.}$$

这里 F_{peak} 是单卡理论峰值, u 是实际有效吞吐与峰值之比, G 是卡数, D_{train} 是理想化训练天数。通信、故障恢复、评估和数据停顿若未反映在 u 中, 实际时间还会更长。

1.2 8 张 80GB H100 最多能装多少参数

若参数与梯度用 BF16, 每项各 2 bytes; AdamW 的一阶矩和二阶矩用 FP32, 各 4 bytes, 则不计激活时每个参数需要

$$b_p = 2 + 2 + 4 + 4 = 12 \text{ bytes.}$$

八张 80GB GPU 的理想参数上限为

$$P_{\text{max}} = \frac{8 \times 80 \times 10^9}{12} \approx 53.3 \times 10^9.$$

其中 b_p 是参数、梯度和优化器状态的合计字节数, P_{max} 是忽略激活、临时缓冲、碎片和通信副本后的理论上界。

```

71 def motivating_questions():
72     total_flops: How long would it take to train a 70B parameter model on 15T tokens on 1024 B100s?
73     flops_per_day = 6 * 70e9 * 15e12 * 24 * 60 * 60 = 4.377e+22
74     days = total_flops / flops_per_day = 1979e12 / 4.377e+22 = 143.9266
75     h100_bytes = 80,000,000,000
76     bytes_per_parameter = 12 # 2 for parameters, 2 for gradients, 4 for AdamW first moment, 4 for second moment
77     num_parameters = total_flops / flops_per_day = 53,333,333,333.333
78
79     Question: What's the largest model that you can train on 8 H100s using AdamW (naively)?
80     h100_bytes = 80e9
81     bytes_per_parameter = 2 + 2 + (4 + 4) # parameters, gradients, optimizer state
82     num_parameters = (h100_bytes * 8) / bytes_per_parameter
83     Caveat: activations are not accounted for (depends on batch size and sequence length).
84
85     This is a rough back-of-the-envelope calculation.
86
87

```

图 2: 草算结果: 约 144 天与约 53.3B 参数; 后者尚未计入激活, 因此只能视为上界。²

数量级估算的边界

草算的价值不是给出精确采购数字, 而是暴露主导项。把激活、序列长度、张量并行副本、主权重、通信缓冲和检查点加入后, 显存结论会改变; 把数据加载、评估与故障加入后, 墙钟结论也会改变。先有可解释的上界, 再逐层补项。

²视频画面时间区间: 00:03:45–00:04:00。

1.3 本章小结

训练预算可以先压缩成两条公式：计算近似 $6PT$ ，静态训练状态约 $12P$ bytes。它们不是最终答案，却能在几分钟内排除明显不可行的规模，并指出下一步必须细算的激活、通信和利用率。

2 张量是资源核算的统一对象

参数、数据、梯度、优化器状态和中间激活看似用途不同，在 PyTorch 中都由张量表示。一个张量至少要从四个角度理解：**形状**决定有多少元素，**dtype** 决定每个元素多少字节，**device** 决定存放位置，**运算图**决定它是否需要为反向传播保留信息。

```

80 h100_bytes = 80e9
81 bytes_per_parameter = 2 + 2 + (4 + 4) # parameters, gradients, optimizer state
82 num_parameters = (h100_bytes * 8) / bytes_per_parameter
83 Caveat: activations are not accounted for (depends on batch size and sequence length).
84
85 This is a rough back-of-the-envelope calculation.
86
87
88 def tensors_basics():
89     Tensors are the basic building block for storing everything:
90     • parameters, gradients, optimizer state, data, activations.
91
92     Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]
93     [DeepSeek v3.2 model on Hugging Face]
94
95     x = torch.zeros(4) # vector
96     x = torch.zeros(4, 8) # matrix

```

图 3：张量承载训练的全部核心状态：数据、参数、梯度、优化器状态和激活。³

2.1 秩、形状与语义维度

向量是 rank-1 张量，矩阵是 rank-2 张量；Transformer 中常见 rank-4 形状

$$X \in \mathbb{R}^{B \times S \times H \times D_h}.$$

其中 B 是 batch size， S 是序列长度， H 是注意力头数， D_h 是每头维度。维度不仅是整数位置，还承担语义；若只记“-2”、“-1”，代码很容易在重排、广播或收缩时悄悄出错。

2.2 显存等于元素数乘元素宽度

连续张量的基础显存核算为

$$M(X) = \text{numel}(X) \times \text{element_size}(X).$$

³视频画面时间区间：00:05:15–00:05:30。

其中 $M(X)$ 是字节数， $\text{numel}(X)$ 是元素总数， $\text{element_size}(X)$ 是 dtype 的单元字节数。形状 (4,8) 的 FP32 张量有 32 个元素，每元素 4 bytes，共 128 bytes。

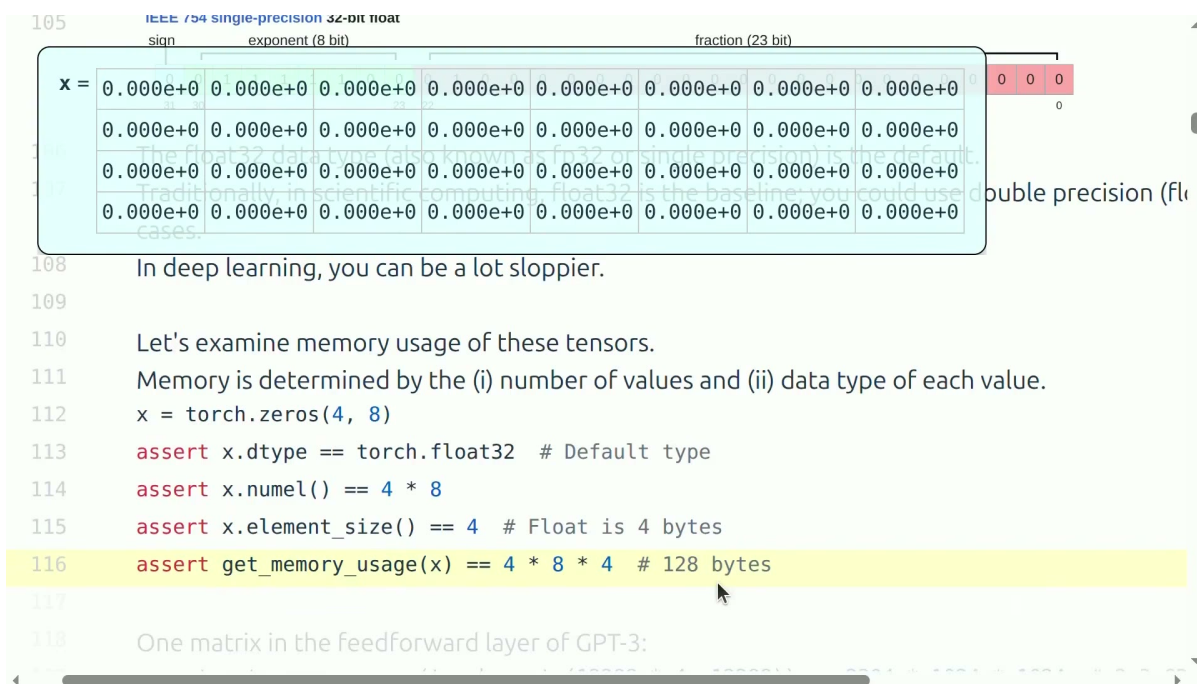


图 4: FP32 示例：显存由元素数与单元字节数共同决定。⁴

以 GPT-3 前馈层的一块权重为例，若矩阵形状是 $12288 \times (4 \cdot 12288)$ ，FP32 存储量约为

$$12288 \times 49152 \times 4 \approx 2.42 \times 10^9 \text{ bytes} \approx 2.25 \text{ GiB.}$$

这只是一个矩阵；大模型中大量权重、梯度与状态会同时驻留。因此，dtype 不是代码细节，而是模型能否放入设备的一级设计变量。

容量核算还要留出非张量开销

真实峰值通常高于“所有张量字节数之和”：算子会申请 workspace，分配器会保留缓存，张量布局可能产生临时 contiguous 副本，分布式库还需要通信桶。工程预算应保留余量，并用峰值显存测量验证静态表格。

2.3 本章小结

资源核算的统一入口是张量清单。先为每个张量写出语义形状、dtype、device 和生命周期，再计算元素数与字节数；这样显存问题才能从“感觉不够”变成可审计的账本。

3 数值格式：容量、范围与分辨率的三角折中

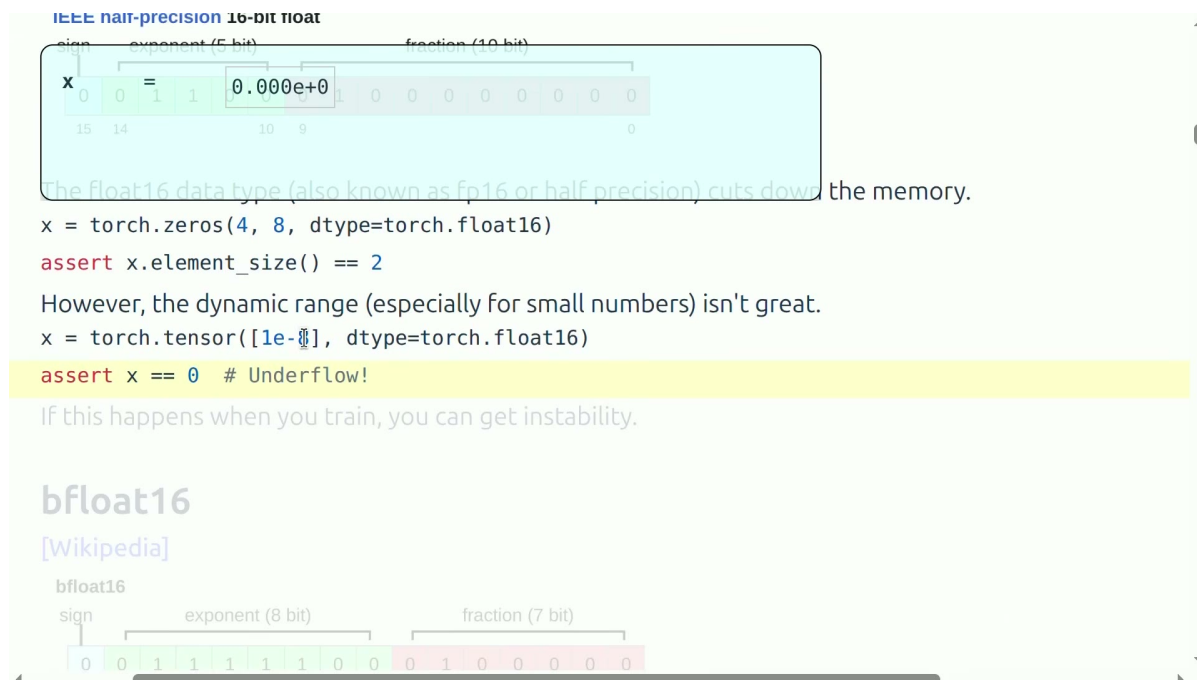
浮点数通常由符号位、指数位和尾数组成。指数位主要控制动态范围，尾数位主要控制相邻可表示数的分辨率。减少位宽能降低显存与带宽，并可能提高 Tensor Core 吞吐，但会增加下溢、上溢和舍入误差。

⁴视频画面时间区间：00:07:30–00:07:45。

格式	符号位	指数位	尾数位	典型字节数
FP32	1	8	23	4
FP16	1	5	10	2
BF16	1	8	7	2
FP8 E4M3	1	4	3	1
FP8 E5M2	1	5	2	1
NVFP4	约 4 bits/value, 另配分块尺度			0.5 (未计尺度)

3.1 FP16: 省一半显存，但小数可能直接归零

FP16 只有 5 个指数位，动态范围明显小于 FP32。课件将 10^{-8} 转为 FP16 后得到 0；若梯度或更新量在训练中发生这种下溢，优化器可能失去有效信号。



IEEE half-precision 16-bit float

sign (1 bit) | exponent (5 bits) | fraction (10 bits)

x = 0.000e+0

The float16 data type (also known as fp16 or half precision) cuts down the memory.

```
x = torch.zeros(4, 8, dtype=torch.float16)
assert x.element_size() == 2
```

However, the dynamic range (especially for small numbers) isn't great.

```
x = torch.tensor([1e-8], dtype=torch.float16)
assert x == 0 # Underflow!
```

If this happens when you train, you can get instability.

bfloat16

[Wikipedia]

bfloat16

sign (1 bit) | exponent (8 bits) | fraction (7 bits)

图 5: FP16 下溢示例: 10^{-8} 被量化为 0, 低精度会直接改变训练动力学。⁵

3.2 BF16: 保留 FP32 的范围，牺牲分辨率

BF16 与 FP32 都有 8 个指数位，因此动态范围接近；但 BF16 只有 7 个尾数位，数值间隔更粗。深度学习的随机性和冗余常能容忍较低分辨率，而上溢、下溢往往更危险，所以 BF16 成为训练中的常用折中。

⁵视频画面时间区间: 00:09:15–00:09:30。

bfloat16



Google Brain developed bfloat (brain floating point) in 2018 to address this issue.

bfloat16 uses the same memory as float16 but has the same dynamic range as float32!

The only catch is that the resolution is worse, but this matters less for deep learning.

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
```

```
assert x != 0 # No underflow!
```

Let's compare the dynamic ranges and memory usage of the different data types:

```
float32_info = torch.finfo(torch.float32)
float16_info = torch.finfo(torch.float16)
bfloat16_info = torch.finfo(torch.bfloat16)
```

Implications on training:

- Training with float32 works, but requires lots of memory.

图 6: BF16 用 8 位指数保留动态范围；代价是更少尾数位和更低分辨率。⁶

3.3 混合精度：不同状态承担不同数值责任

课程给出的常见配置是：参数、激活和梯度使用 BF16，优化器累计状态使用 FP32。前者占据大部分容量与带宽，适合降精度；后者跨很多训练步累积，舍入误差更敏感。PyTorch AMP 会在安全的算子（如矩阵乘法）中自动降精度，而对指数等敏感算子保留更高精度。

```
142 • Training with float32 works, but requires lots of memory.
143 • Training with fp8, float16 and even bfloat16 is risky, and you can get instability.
144
145 Mixed precision training [Micikevicius+ 2017]
146 • Use bf16 for parameters, activations, and gradients
147 • Use fp32 for optimizer states
148
149 Pytorch has an automatic mixed precision (AMP) library. [docs]
150 Tries to cast things into bf16 when safe (matmuls, not exp).
```

fp8

In 2022, fp8 was standardized, motivated by machine learning workloads [primer](#).

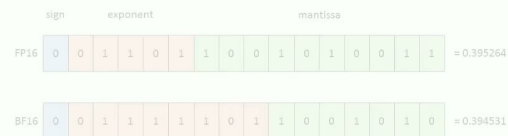


图 7: 混合精度分工：大体量训练张量用 BF16，长期累计的优化器状态用 FP32。⁷

⁶视频画面时间区间：00:10:45–00:11:00。

3.4 FP8 与 NVFP4：位宽继续下降后必须引入尺度管理

H100 支持 FP8 E4M3 与 E5M2：前者尾数更多，后者范围更大。它们的选择取决于张量分布与算子位置，不能简单把所有 BF16 张量统一转换为 FP8。

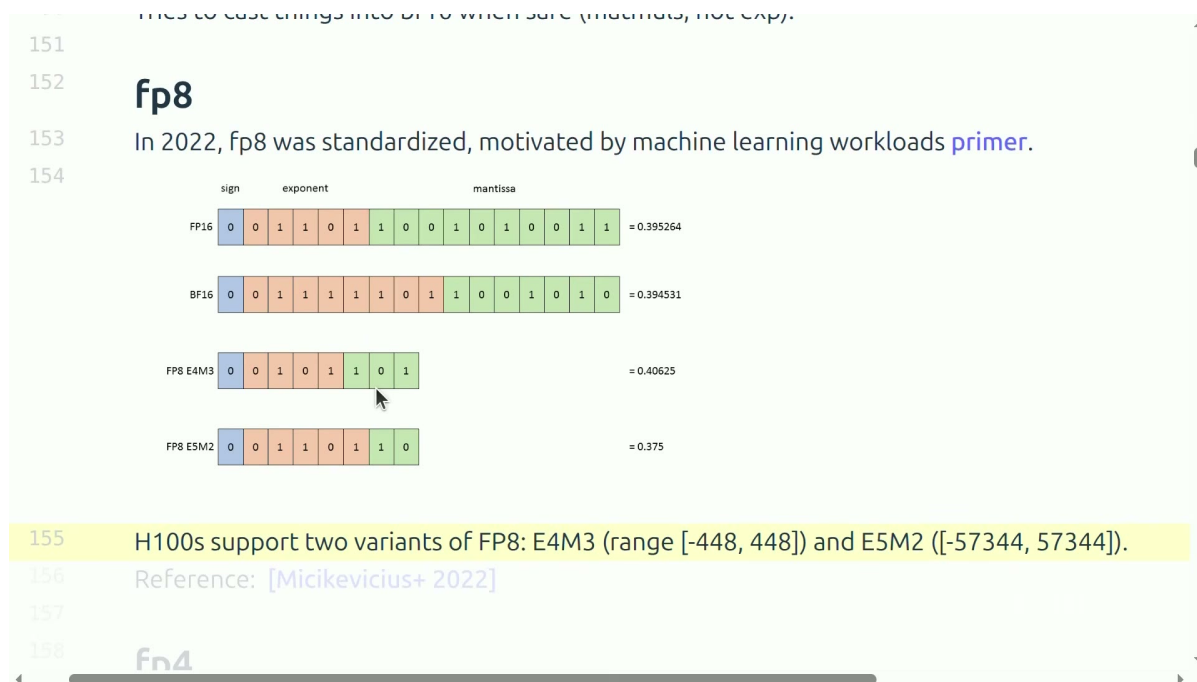


图 8：FP8 两种格式在指数位和尾数位之间重新分配预算。⁸

NVFP4 每个值只有 4 bits，可表示的基础离散值很少，因此按 block 额外存储 scale factor。块内数共享尺度：扩大了有效范围，却也让相邻值的缩放耦合。低位宽的系统收益依赖硬件内核、分块策略、校准和误差控制，而不仅是文件变小。

⁷视频画面时间区间：00:12:45–00:13:00。

⁸视频画面时间区间：00:13:30–00:13:45。

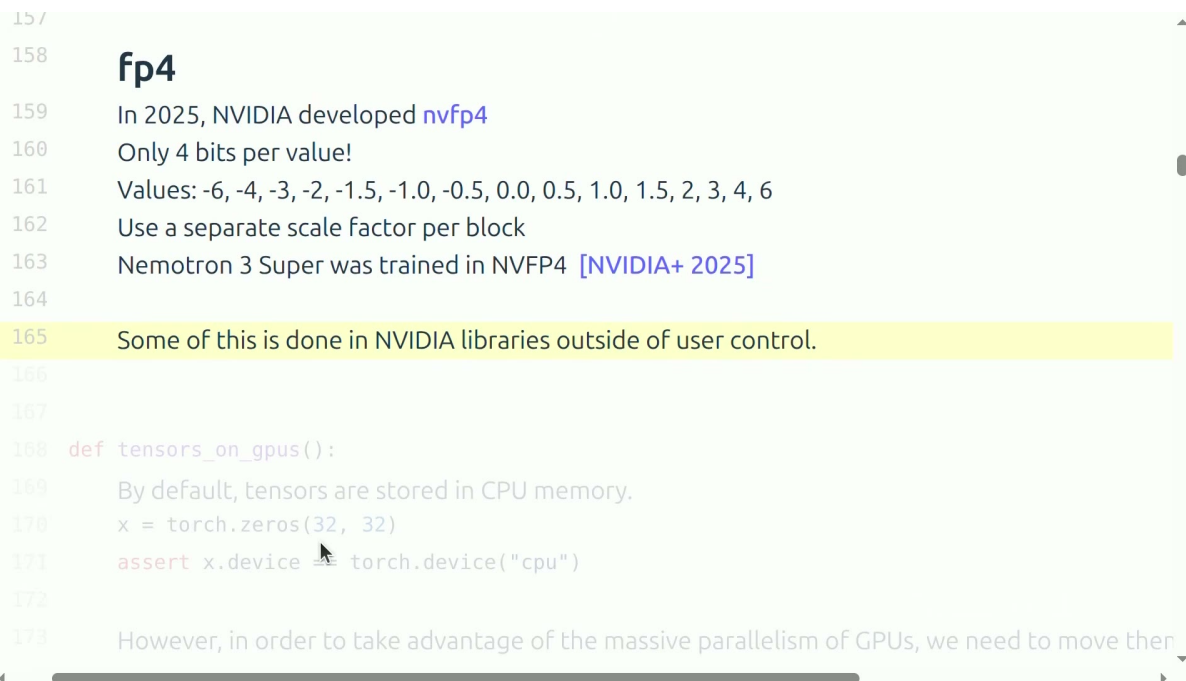


图 9: NVFP4 通过离散 4-bit 值与分块尺度共同表示更宽范围。⁹

位宽选择必须同时验收三项

一是质量与稳定性，二是峰值显存和数据搬运，三是实际 kernel 吞吐。若硬件没有对应低精度路径，或者频繁量化、反量化与尺度归约占据时间，理论上的位宽优势不会自动变成墙钟优势。

3.5 本章小结

FP16、BF16、FP8 与 FP4 不是一条单调“越低越好”的阶梯。指数位控制范围、尾数位控制分辨率、分块尺度补偿超低位宽；混合精度的本质是让每类状态使用刚好足够的数值预算。

4 设备放置与具名张量运算

4.1 CPU 与 GPU 之间的移动也是成本

PyTorch 默认把张量创建在 CPU 内存。要利用 GPU 的并行计算，输入必须进入 GPU DRAM；CPU RAM 与 GPU DRAM 之间通常通过 PCIe 或更快互连搬运。若每个小算子都来回移动数据，即使 GPU 算得很快，端到端性能仍会被传输吞没。

⁹视频画面时间区间：00:14:45–00:15:00。

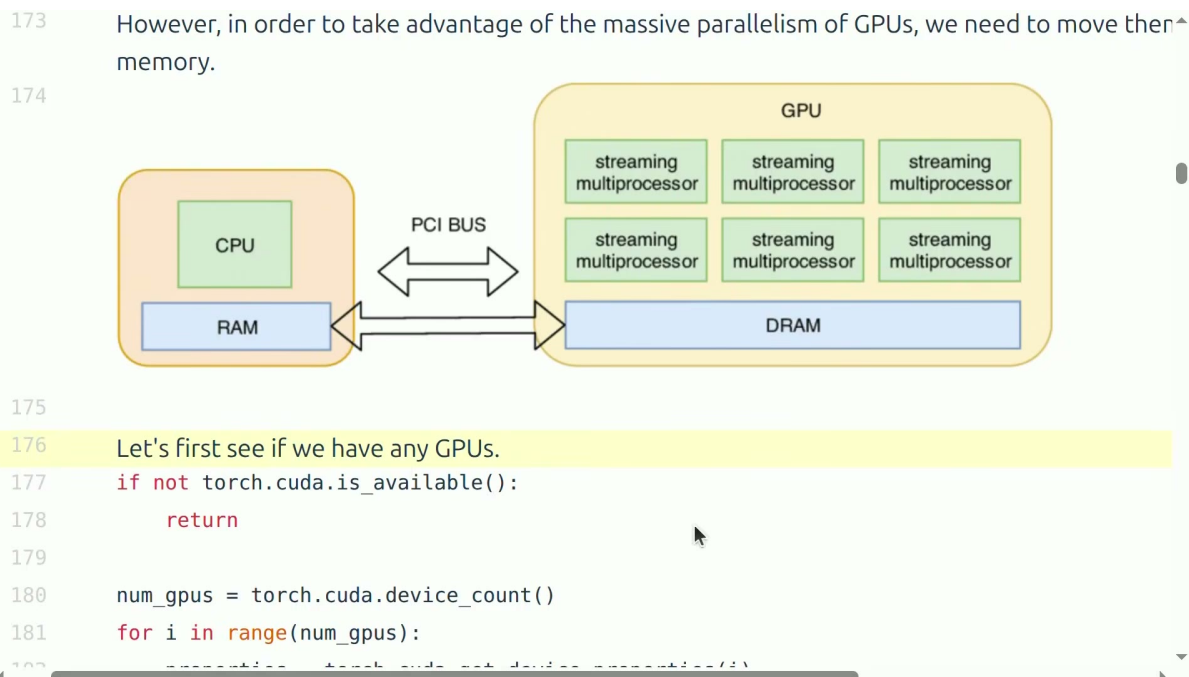


图 10: CPU 与 GPU 拥有独立内存; 跨 PCIe 搬运是显式资源成本。¹⁰

设备核算至少要回答: 张量在哪里创建、在哪个设备执行、何时复制、复制是否能与计算重叠。理想路径是尽早把批数据送入目标设备, 并让中间状态留在设备上, 直到必须输出。

4.2 Einsum: 把维度语义写进运算

传统写法常依赖位置索引:

```
1 z = x @ y.transpose(-2, -1)      # batch seq1 seq2
```

用 einops 的具名维度写法可明确表达收缩关系:

```
1 z = einsum(x, y,
2           "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

输出中未出现的 `hidden` 维会被求和; 共同保留的 `batch` 维则参与批量广播。省略号还可代表任意数量的前导批维。

¹⁰视频画面时间区间: 00:17:30–00:17:45。

```

232 Let's try a more complex example...
233
234 x = torch.ones(2, 3, 4) # batch seq1 hidden
235 y = torch.ones(2, 3, 4) # batch seq2 hidden
236
237 # Old way
238 z = x @ y.transpose(-2, -1) # batch seq1 seq2
239
240 # New (einops) way
241 z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
242 Dimensions that are not named in the output are summed over.
243
244 # Or can use `...` to represent broadcasting over any number of dimensions
245 z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
246
247
248 def einops_reduce():
249     You can reduce a single tensor via some operation (e.g., sum, mean, max, min).

```

图 11: 具名 Einsum 明确写出批维、序列维和被收缩的隐藏维。¹¹

4.3 Reduce 与 Rearrange: 把形状变换变成类型检查

`reduce` 用名字指定被聚合的维度; `rearrange` 可以拆开或合并复合维度。例如把总隐藏维 d_{total} 拆成头数 H 与每头宽度 d_h , 逐头做线性变换, 再合并回去:

```

1 x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=H)
2 x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
3 x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

```

¹¹ 视频画面时间区间: 00:22:30-00:22:45。

```

258
259 def einops_rearrange():
260     Sometimes, a dimension represents two dimensions
261     ...and you want to operate on one of them.
262
263     x = torch.ones(3, 8) # seq total_hidden
264     ...where total_hidden is a flattened representation of heads * hidden1
265     w = torch.ones(4, 4) # hidden1 hidden2
266
267     # Break up `total_hidden` into two dimensions (`heads` and `hidden1`
268     x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
269
270     # Perform the transformation by `w`
271     x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
272
273     # Combine `heads` and `hidden2` back together
274     x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

```

图 12: Rearrange 将扁平隐藏维拆成多头结构，运算后再合并。¹²

具名维度不是语法糖而是局部规范

当形状复杂时，维度名相当于轻量类型系统：它降低轴顺序错误，也让 FLOPs 和显存公式能直接对应代码。推荐在实现旁写出每个张量的语义形状，并在边界处用断言验证。

4.4 本章小结

张量不仅有形状，还驻留在具体设备并经历具体搬运。Einsum、Reduce 和 Rearrange 把维度语义暴露在代码中，使正确性检查、FLOPs 计数和通信分析共享同一套符号。

5 FLOPs: 从矩阵形状推导计算量

FLOP 表示一次浮点加法或乘法；FLOP/s 表示硬件每秒能完成多少浮点运算。二者读音相同却含义不同：前者是工作量，后者是速度。硬件表中的 FLOP/s 还依赖 dtype、稀疏假设和矩阵形状，不能脱离条件比较。

¹²视频画面时间区间：00:26:00–00:26:15。

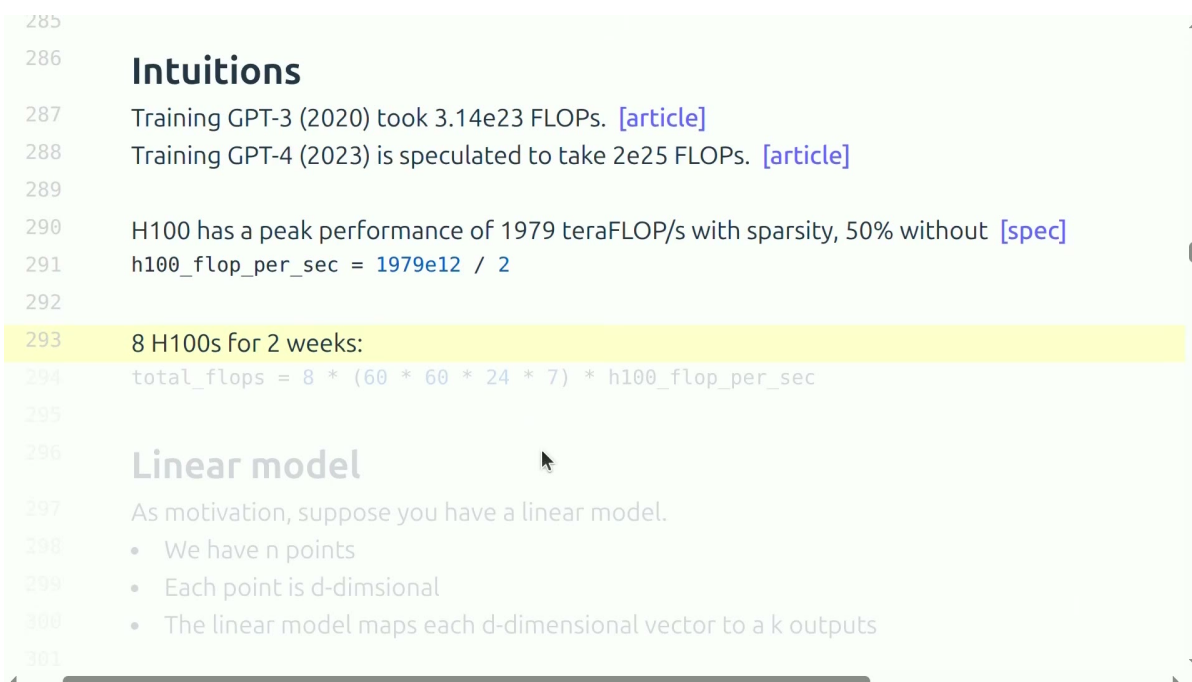


图 13: 训练总 FLOPs 与硬件峰值 FLOP/s 是两个量; H100 稠密值需去掉稀疏加速假设。¹³

5.1 矩阵乘法为何是 2BDK

令

$$X \in \mathbb{R}^{B \times D}, \quad W \in \mathbb{R}^{D \times K}, \quad Y = XW.$$

每个输出 Y_{bk} 是长度 D 的点积, 包含约 D 次乘法与 D 次加法; 共有 BK 个输出, 因此

$$C_{\text{matmul}} \approx 2BDK.$$

其中 B 是数据点或 token 数, D 是输入维度, K 是输出维度, C_{matmul} 是浮点运算数。严格来说每个点积有 $D - 1$ 次加法, 但大形状下用 $2D$ 近似足够。

¹³视频画面时间区间: 00:30:00–00:30:15。

```

02     if torch.cuda.is_available():
03         B = 16384 # Number of points
04         D = 32768 # Dimension
05         K = 8192  # Number of outputs
06     else:
07         B = 1024
08         D = 256
09         K = 64
10
11     x = torch.ones(B, D, device=cuda_if_available())
12     w = torch.randn(D, K, device=cuda_if_available())
13     y = x @ w
14     We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.
15     actual_num_flops = 2 * B * D * K
16
17     FLOPs of other operations
18     - Elementwise operation on a m x n matrix requires O(m * n) FLOPs

```

图 14: 矩阵乘法中每个 (b, d, k) 三元组对应一次乘法和一次加法。¹⁴

5.2 其他算子通常不是主导 FLOPs，却可能主导时间

对 $B \times K$ 张量做逐元素激活通常是 $O(BK)$ ；两个同形矩阵相加也是 $O(BK)$ 。与 $O(BDK)$ 的大矩阵乘法相比，它们的 FLOPs 很少。但逐元素算子对每个字节只做少量工作，常受内存带宽限制。由此得到重要区分：**少 FLOPs 不代表少时间**。

统一计数口径

不同资料有时把 fused multiply-add 计作一次指令，有时计作两个 FLOPs。训练核算通常按一乘一加等于 2 FLOPs。比较论文、硬件表和 profiler 时，先确认口径、稀疏性和 dtype。

5.3 本章小结

计算量可以从张量形状机械推导：矩阵乘法约为 $2BDK$ ，逐元素运算约为元素数。FLOPs 是必要账本，但不能单独预测墙钟时间，因为访存和硬件利用率尚未进入公式。

6 正确基准测试与 MFU

6.1 GPU 异步执行为何会让计时失真

CPU 发起 CUDA kernel 后通常立即返回，GPU 在后台执行。如果只用普通墙钟包围一次调用，测到的可能只是 launch 时间。正确的微基准要在计时区间前后同步 GPU，先预热，再重复多次并报告稳定统计量。

¹⁴视频画面时间区间：00:32:00–00:32:15。

```

353
354 def time_matmul(a: torch.Tensor, b: torch.Tensor) -> float:
355     """Return the number of seconds required to perform `a @ b`."""
356
357     # Wait until previous CUDA threads are done
358     if torch.cuda.is_available():
359         torch.cuda.synchronize()
360
361     def run():
362         # Perform the operation
363         a @ b
364
365     # Wait until CUDA threads are done
366     if torch.cuda.is_available():
367         torch.cuda.synchronize()
368
369     # Time the operation `num_trials` times
370     num_trials = 5

```

图 15: CUDA 微基准在执行前后同步，避免把异步 launch 误当成完整执行时间。¹⁵

若已知操作量 C 与实测时间 t ，有效吞吐为

$$F_{\text{actual}} = \frac{C}{t}.$$

其中 F_{actual} 是实测 FLOP/s， C 是已执行 FLOPs， t 是同步后的秒数。

6.2 MFU：理论峰值到底兑现了多少

Model FLOPs Utilization 定义为

$$\text{MFU} = \frac{F_{\text{actual}}}{F_{\text{promised}}}.$$

其中 F_{promised} 是给定设备、dtype 和稠密性条件下的理论峰值。课程把 $\text{MFU} \geq 0.5$ 视为通常已经不错的结果；但它忽略通信与其他开销的细分，因此是总体现象指标，不是根因诊断。

¹⁵视频画面时间区间：00:35:30–00:35:45。

```

329 Let us time it!
330 actual_time = time_matmul(x, w)
331 actual_flop_per_sec = actual_num_flops / actual_time
332
333 Each GPU has a specification sheet that reports the peak performance.
334 • Example: [H100 spec]
335 Note that the FLOP/s depends heavily on the data type!
336 promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
337
338 Model FLOPs utilization (MFU)
339
340 Definition: (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]
341 mfu = actual_flop_per_sec / promised_flop_per_sec
342 Usually, MFU of  $\geq 0.5$  is quite good (and will be higher if matmuls dominate)
343
344 Summary

```

图 16: MFU 将实测 FLOP/s 与对应 dtype 的理论峰值相除。¹⁶

MFU 低可能来自矩阵太小、形状不友好、访存受限、kernel launch 过多、同步、通信、数据加载或负载不均。定位时要把握端到端 MFU 逐层拆成算子时间、数据移动和空闲时间。

一个可复用的基准流程

固定形状与 dtype；预热直到缓存和编译稳定；在 GPU 前后同步；重复测量并报告中位数与分位数；同时记录 FLOPs、峰值显存和硬件型号；最后用 profiler 确认没有隐式复制或不期望的同步。

6.3 本章小结

GPU 基准测试首先要处理异步性。MFU 把计算量和墙钟时间连接起来，但它只是利用率结果；要解释低 MFU，必须继续分析计算与内存之间的瓶颈。

7 算术强度：计算与数据搬运谁更慢

一个 GPU 算子的抽象流程是：从显存读取输入，在计算单元执行操作，再把输出写回显存。理想情况下计算和搬运可以重叠，则时间下界为

$$t_{\min} = \max \left(\frac{Q}{B_w}, \frac{C}{F_{\text{peak}}} \right).$$

其中 Q 是搬运字节数， B_w 是显存带宽 (bytes/s)， C 是 FLOPs， F_{peak} 是峰值 FLOP/s。

¹⁶ 视频画面时间区间：00:37:15–00:37:30。

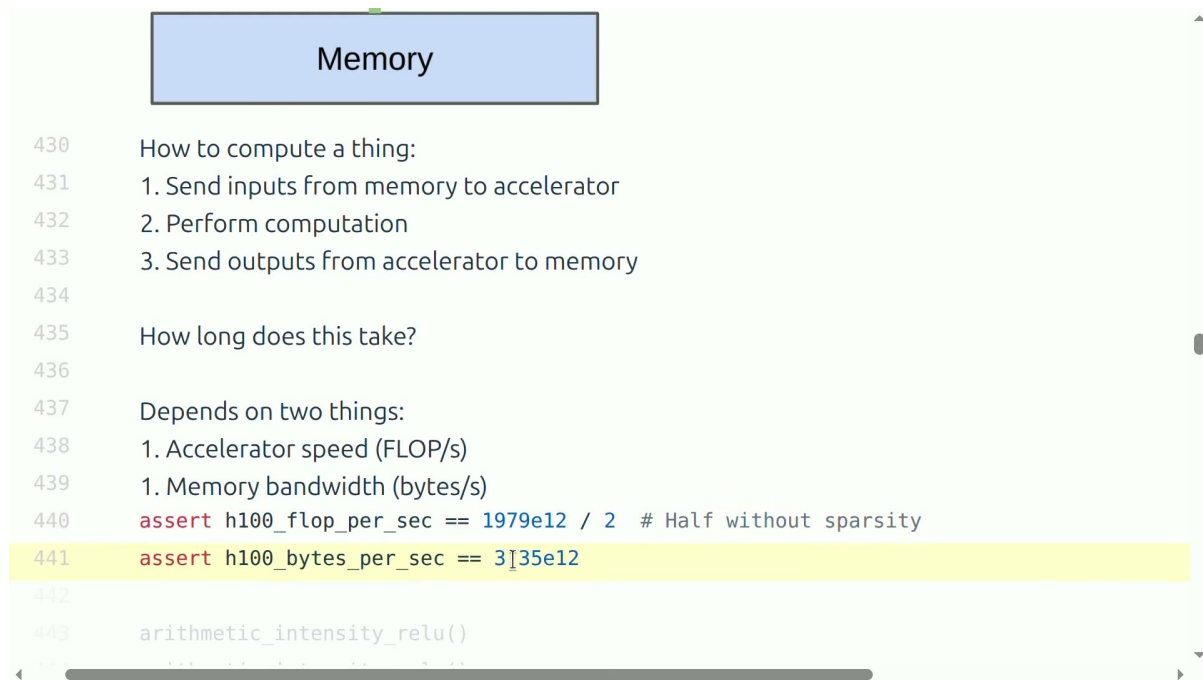


图 17: 算子时间由显存搬运与计算两条路径中更慢的一条决定。¹⁷

7.1 ReLU: FLOPs 很少，时间却由带宽决定

对 n 个 BF16 元素做 ReLU，读输入 $2n$ bytes、写输出 $2n$ bytes，若每元素近似一次比较，则

$$Q_{\text{ReLU}} = 4n, q_{\text{quad}} C_{\text{ReLU}} \approx n.$$

在课件的 H100 数值下，通信时间约 1.252×10^{-6} 秒，而计算时间约 1.060×10^{-9} 秒，前者大约高三个数量级。

¹⁷ 视频画面时间区间：00:41:45–00:42:00。

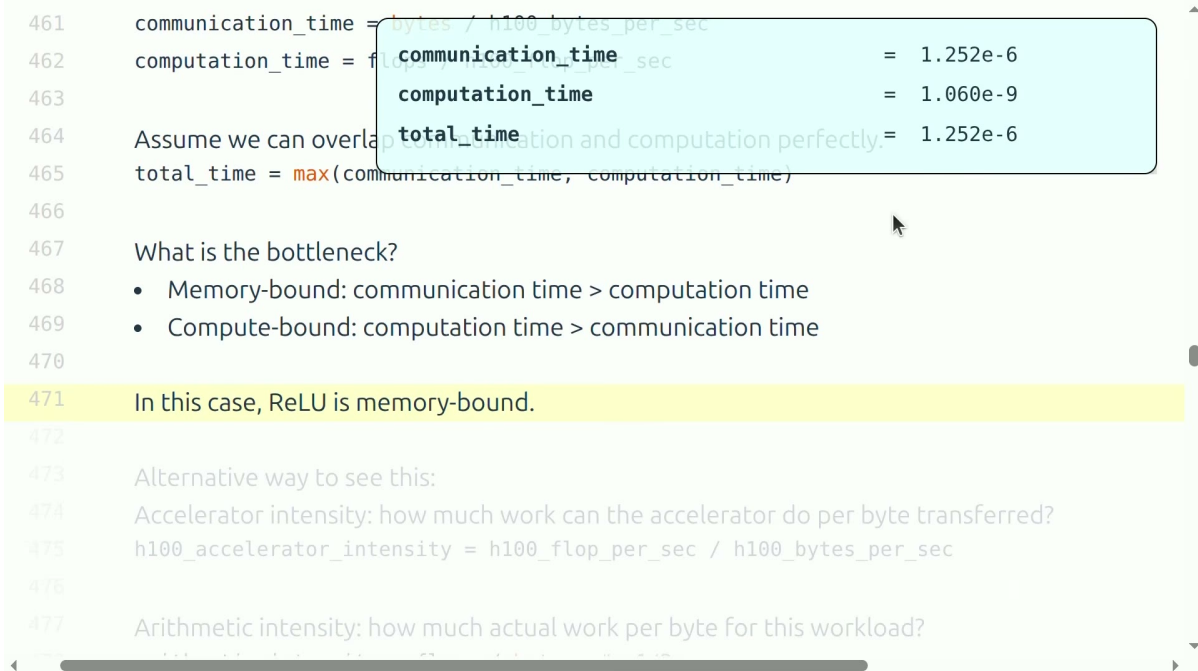


图 18: ReLU 示例中显存通信时间远大于计算时间，因此是 memory-bound。¹⁸

7.2 工作负载强度与硬件临界强度

定义工作负载的算术强度

$$I = \frac{C}{Q} \quad (\text{FLOPs/byte}),$$

以及硬件临界强度

$$I_* = \frac{F_{\text{peak}}}{B_w}.$$

其中 I 表示每搬一个字节做多少工作， I_* 表示硬件要吃满计算单元所需的最低工作密度。若 $I < I_*$ ，算子受带宽限制；若 $I > I_*$ ，算子受计算限制。

课件使用 H100 稠密 BF16 峰值 989.5 TFLOP/s 和显存带宽 3.35 TB/s，得到

$$I_* \approx 295.37 \text{ FLOPs/byte}.$$

ReLU 的 $I \approx n/(4n) = 0.25$ ，远低于临界值。

¹⁸ 视频画面时间区间：00:45:45–00:46:00。

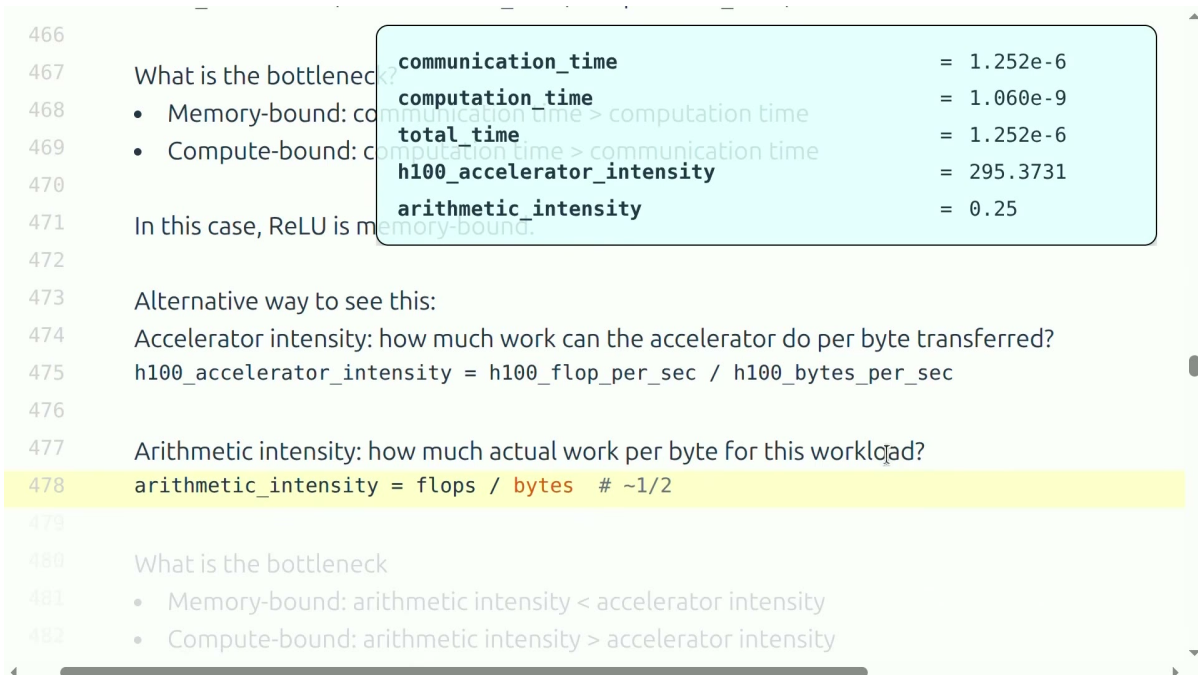


图 19: ReLU 的工作负载强度约 0.25 FLOPs/byte, 而 H100 临界强度约 295。¹⁹

GeLU 虽包含多项式和双曲函数近似, 课件按每元素约 20 FLOPs 估算, 因此 $I \approx 20n/(4n) = 5$ 。它比 ReLU 做更多计算, 却仍远低于 I_* ; 孤立执行时, 两者都主要在等显存。

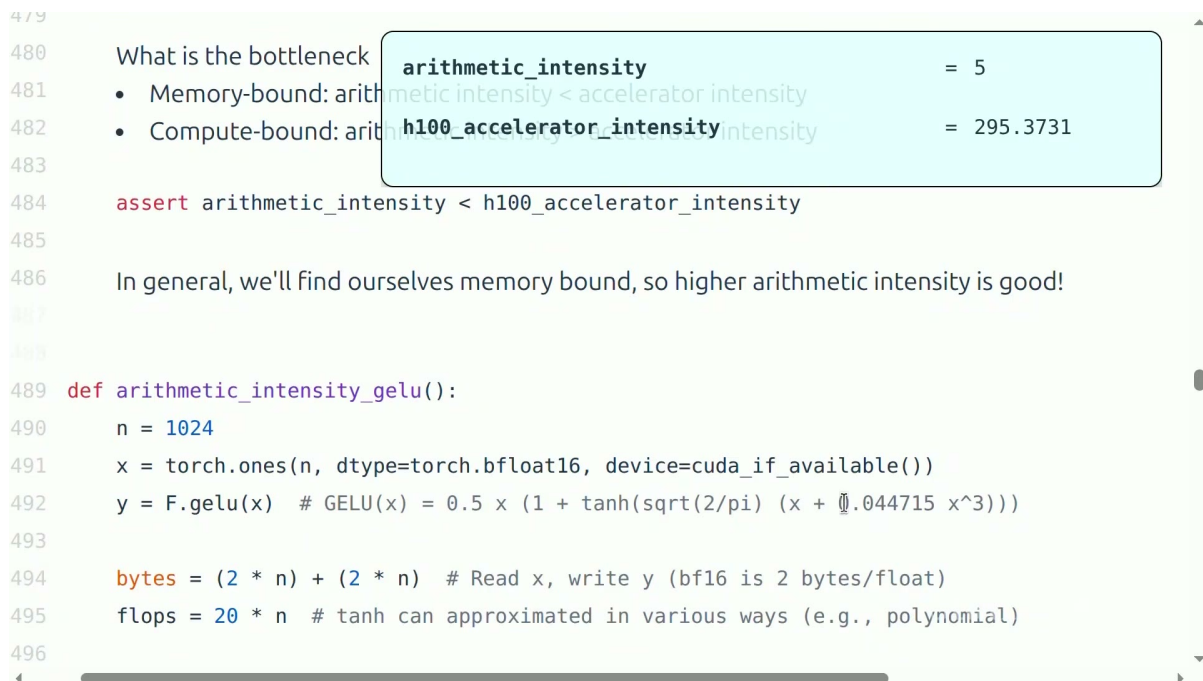


图 20: GeLU 的算术强度约为 5, 明显高于 ReLU, 但仍属于 memory-bound。²⁰

¹⁹视频画面时间区间: 00:47:00–00:47:15。

²⁰视频画面时间区间: 00:49:30–00:49:45。

融合为什么有效

算子融合不一定显著减少理论 FLOPs，却能让中间值留在寄存器或片上存储，减少 HBM 的读写字节 Q 。由于 $I = C/Q$ ，降低 Q 会提高算术强度，也可能减少 kernel launch 和同步开销。

7.3 本章小结

算术强度把 FLOPs 与字节数放进同一坐标。逐元素算子的计算便宜但搬运昂贵，往往 memory-bound；优化重点应是减少中间读写、融合操作并增加数据复用。

8 Roofline：矩阵乘法为何适合训练，矩阵向量为何拖慢推理

Roofline 模型给出的可实现吞吐上界是

$$F_{\text{realized}} \leq \min(F_{\text{peak}}, IB_w).$$

其中水平屋顶 F_{peak} 表示计算上限，斜线 IB_w 表示带宽上限，两者交点的横坐标就是 I_* 。

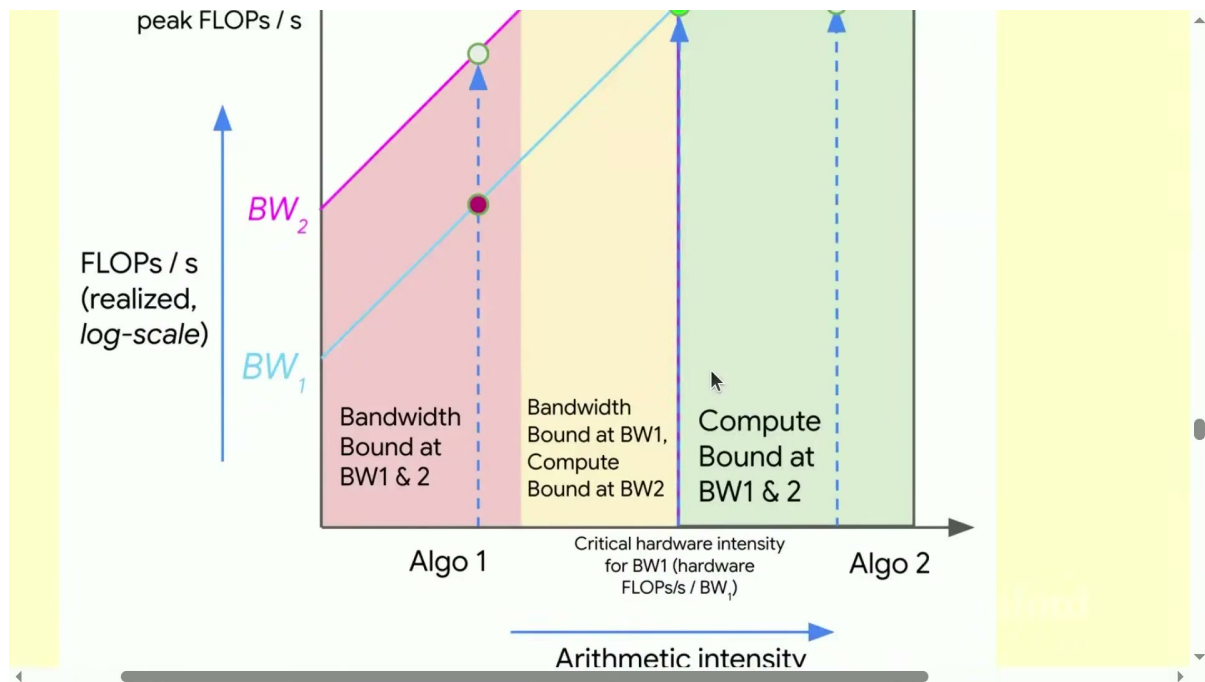


图 21: Roofline 将算术强度映射到可实现 FLOP/s，并划分带宽受限与计算受限区域。²¹

8.1 几类操作的强度对比

对 BF16 大形状忽略低阶项：

- 点积读取两个长度 n 的向量， $I \approx 1/2$ ；
- $n \times n$ 矩阵乘向量，权重只复用一次， $I \approx 1$ ；

²¹视频画面时间区间：00:55:45–00:56:00。

- $n \times n$ 矩阵乘矩阵，输入被反复复用，

$$I_{\text{matmul}} \approx \frac{2n^3}{6n^2} = \frac{n}{3}.$$

这里 n 是方阵边长。若 $n = 1024$ ，矩阵乘法强度约 341 FLOPs/byte，超过课件 H100 的临界值 295，因而有机会成为 compute-bound。

8.2 训练与自回归推理的结构差异

训练把许多 token 组成大矩阵，同一权重被 batch 和序列位置复用，容易形成高强度 GEMM。自回归 decode 每一步常接近矩阵向量乘法，每生成一个 token 都要读取大量权重，复用不足，所以更容易受显存带宽限制。增加 batch 可提高权重复用，但会牺牲单请求延迟并增加 KV cache 压力。

Roofline 还给出 MFU 的理想上界：

$$\text{MFU} \lesssim \min\left(1, \frac{I}{I_*}\right).$$

若算法位于斜坡区，再精巧的计算调度也无法越过带宽屋顶；应先提高复用或减少字节数。

低精度同时移动两条线

降低 dtype 往往减少字节数并提高峰值 FLOP/s，因此既改变工作负载强度 I ，也改变硬件临界强度 I_* 。不能只用“字节减半”预测最终加速，必须重新计算两者并测量对应 kernel。

8.3 本章小结

Roofline 把硬件与算法连接起来。大矩阵乘法通过数据复用提高算术强度，适合吃满训练算力；矩阵向量乘法复用弱，解释了 decode 常被带宽限制。

9 Autograd 与反向传播的计算账本

课程先用由线性层和 ReLU 组成的深网络建立统一例子：每层权重为 $D \times D$ ，输入和激活为 $B \times D$ ，共有 L 层，参数量近似 $P = LD^2$ 。

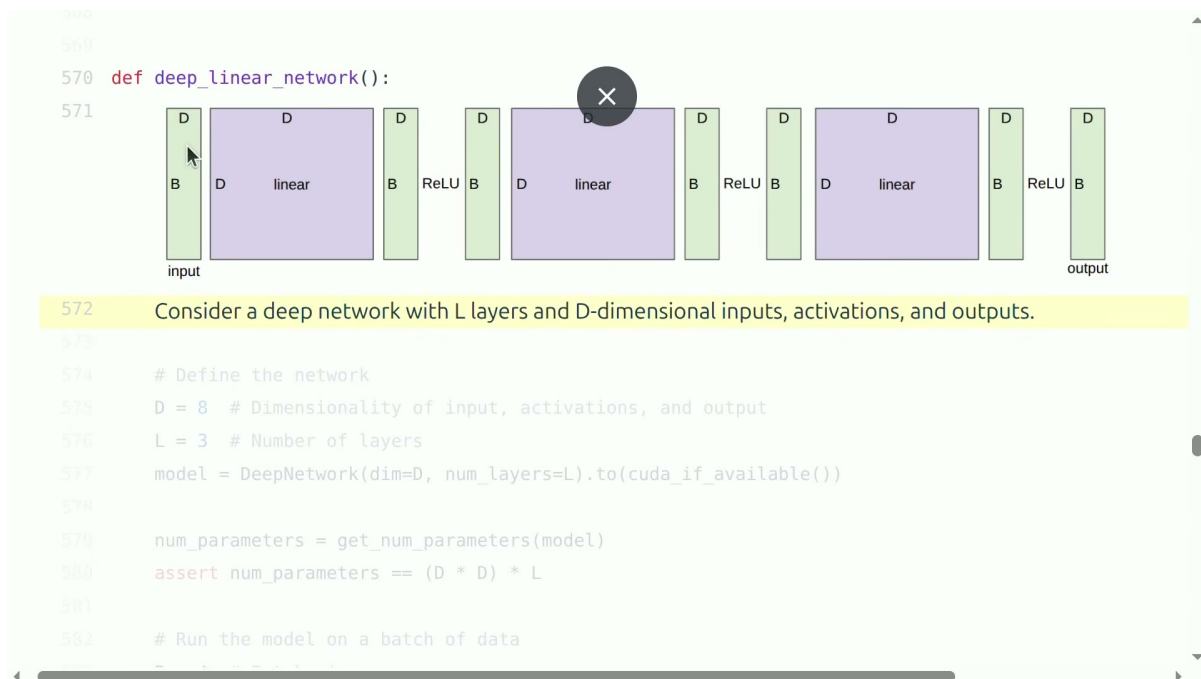


图 22: 运行示例: L 层 $D \times D$ 线性变换与 ReLU 串联。²²

9.1 Autograd 记录什么

若张量设置 `requires_grad=True`, PyTorch 在前向时记录产生结果的操作和依赖, 反向时根据链式法则累积梯度。对

$$\ell = \frac{1}{2}(x^\top w - 5)^2,$$

梯度为

$$\nabla_w \ell = (x^\top w - 5)x.$$

其中 x 是输入, w 是可训练权重, ℓ 是标量损失。课件取 $x = (1, 2, 3)$ 、 $w = (1, 1, 1)$, 残差为 1, 因此 $\nabla_w \ell = (1, 2, 3)$ 。

²²视频画面时间区间: 00:57:45–00:58:00。

```

347 • FLOP/s depends on hardware (B200 >> H100) and data type (bfloat16 >> float32)
348 • Model FLOPs utilization (MFU): (actual FLOP/s) / (promised FLOP/s)
349
350 def gradients_basics():
351     So far, we've constructed tensors (which correspond to either parameters or data) and passed them through
352     operations (forward).
353     Now, we're going to compute the gradient (backward).
354     As a simple example, let's consider the simple linear model:
355     y = 0.5 (x * w - 5)^2
356
357     Forward pass: compute loss
358     x = torch.tensor([1., 2, 3])
359     w = torch.tensor([1., 1, 1], requires_grad=True) # Want gradient
360     pred_y = x @ w
361     loss = 0.5 * (pred_y - 5).pow(2)
362
363     Backward pass: compute gradients
364     loss.backward()
365     """

```

图 23: 简单标量损失展示前向构图与 `loss.backward()` 的梯度计算。²³

9.2 单层反向为何是前向的两倍

对第二层

$$H_2 = H_1 W_2,$$

其中 $H_1, H_2 \in \mathbb{R}^{B \times D}$, $W_2 \in \mathbb{R}^{D \times D}$ 。前向需要

$$C_f \approx 2BD^2.$$

反向要计算两次同量级矩阵乘法:

$$\nabla_{H_1} \ell = \nabla_{H_2} \ell W_2^\top, \quad \nabla_{W_2} \ell = H_1^\top \nabla_{H_2} \ell,$$

所以

$$C_b \approx 4BD^2 \approx 2C_f.$$

这里 C_f 与 C_b 分别是单层前向和反向 FLOPs。

²³视频画面时间区间: 00:59:15–00:59:30。

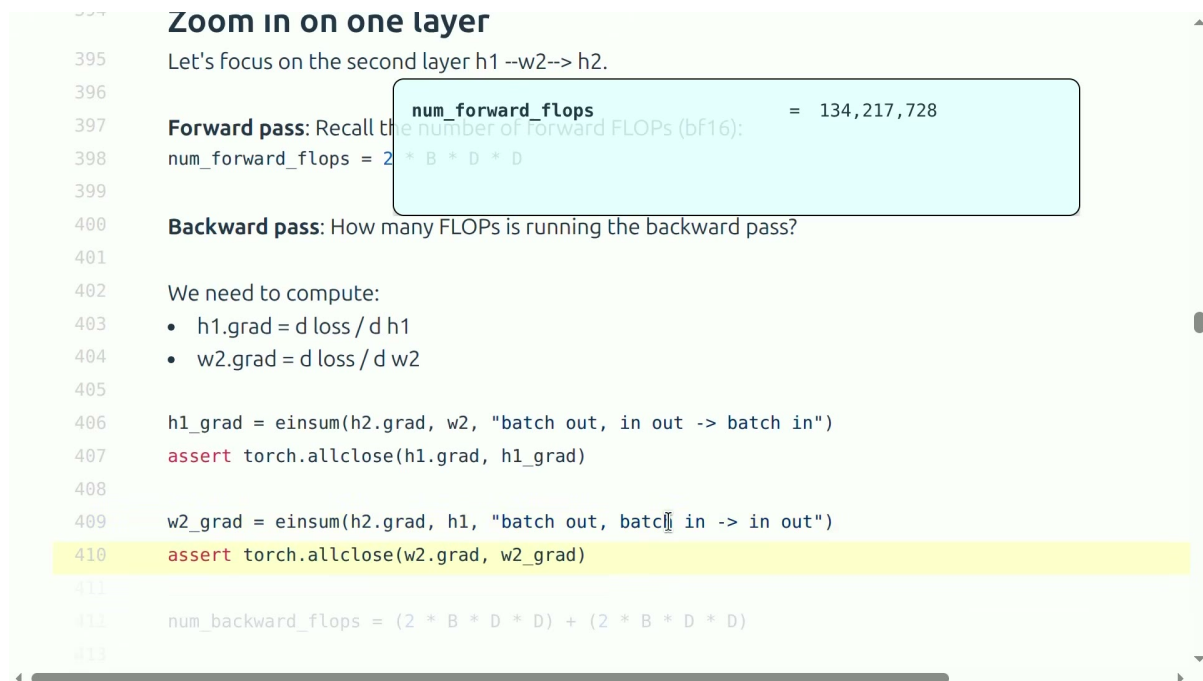


图 24: 单层反向需要计算输入梯度和权重梯度，两者都是矩阵乘法。²⁴

9.3 全网络的 6BP 近似

对所有层求和，若 $P \approx LD^2$ ，则

$$C_{\text{step}} \approx 2BP + 4BP = 6BP.$$

其中 B 表示该步处理的数据点或 token 数， P 是参数量。这一推导对 MLP 精确到主导项，对短上下文 Transformer 也是常用近似；长上下文下注意力的序列二次项会变得重要，需单独加入。

²⁴ 视频画面时间区间：01:04:30–01:04:45。

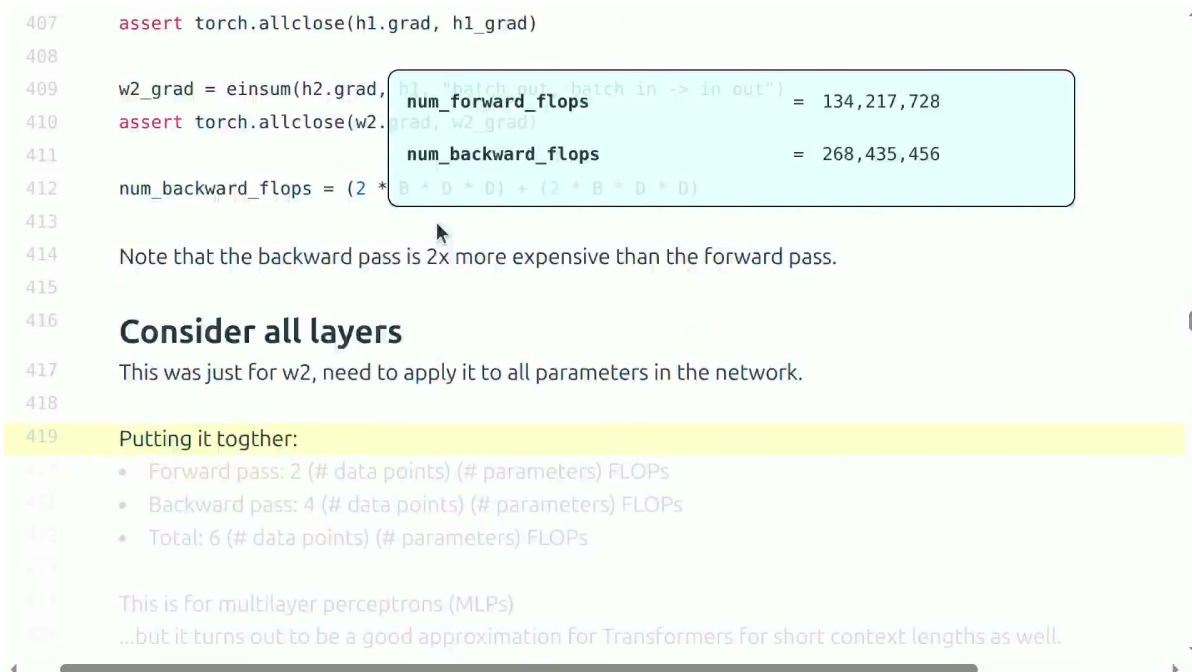


图 25: 前向约 2BP, 反向约 4BP, 训练一步合计约 6BP。²⁵

不要把 6PT 用到所有情形

稀疏 MoE 只有部分参数被激活; 权重共享、嵌入、长上下文注意力和非标准算子会改变比例。6PT 是先验, 不是测量。最终应从真实模型形状逐层计数, 并与 profiler 或训练日志交叉验证。

9.4 本章小结

Autograd 的计算并非黑箱: 线性层反向由输入梯度和权重梯度两次矩阵乘法组成, 因此约为前向两倍。累加到全网络后得到训练规模估算的核心 6BP。

10 训练显存: 静态状态与随 batch 增长的激活

在 BF16 参数与梯度、FP32 Adam 一阶/二阶矩的简化配置中, 显存主项为

$$M_{\text{train}} \approx 2P + 2P + 8P + M_{\text{act}} = 12P + M_{\text{act}}.$$

其中 $2P$ 是参数, 另一个 $2P$ 是梯度, $8P$ 是两个 FP32 优化器状态, M_{act} 是反向所需激活。若还保留 FP32 master weights, 需再加 $4P$; 不同优化器和分片策略也会改变系数。

²⁵ 视频画面时间区间: 01:05:45–01:06:00。


```

643
644 Memory
645
646 # Parameters
647 parameter_memory = 2 * (D * D * L) # (2 bytes for bf16)
648
649 # Activations
650 activation_memory = 2 * B * D * L # (2 bytes for bf16)
651
652 # Gradients
653 gradient_memory = 2 * parameter_memory # (2 bytes for bf16)
654
655 # Optimizer states
656 optimizer_state_memory = 4 * parameter_memory # (4 bytes for fp32)
657 Customary to use fp32 for stability (accumulating averages over powers over many steps)
658 4 bytes/parameter for storing second moments, Adam requires 8 bytes/parameter for storing first and second
659 moments
660
661 # Putting it all together

```

图 26: 训练显存账本: 参数、激活、梯度和优化器状态具有不同 dtype 与生命周期。²⁶

对课堂的 L 层、宽度 D 的简化网络, 若每层保存一个 $B \times D$ BF16 激活, 则

$$M_{\text{act}} \approx 2BDL \text{ bytes.}$$

其中 B 是 batch size, D 是激活宽度, L 是层数。真实 Transformer 还会保存 Q/K/V、注意力概率、归一化统计或算子专用中间量, 系数通常更大。

10.1 生命周期决定峰值, 而不是简单总和

参数和优化器状态长期驻留; 梯度在反向逐步产生; 激活在前向积累、反向逐层释放; 临时 workspace 只在算子执行期间存在。峰值显存必须按时间线计算, 不能把从未同时存在的临时量全部相加, 也不能漏掉在峰值时重叠的通信缓冲。

显存优化的三个方向

减少每元素字节数 (低精度), 减少同时驻留的元素数 (分片、微批、检查点), 或缩短生命周期 (及时释放、原地或融合)。每一种都可能以计算、通信、精度或实现复杂度为代价。

10.2 本章小结

训练显存至少包含静态的参数、梯度和优化器状态, 以及随 batch、序列长度和层数增长的激活。准确预算要同时考虑 dtype、张量形状和生命周期。

²⁶ 视频画面时间区间: 01:10:45–01:11:00。

11 用计算换显存：梯度累积与激活检查点

11.1 梯度累积：把一个大 batch 切成多个微批

若目标 batch 为 B ，将其分成 K 个 micro-batch，每个大小 $b = B/K$ 。依次前向和反向，梯度不清零；处理完 K 个微批后再更新参数。于是活跃激活大致从与 B 成正比降为与 b 成正比：

$$M_{\text{act, live}} \propto bDL = \frac{B}{K}DL.$$

其中 K 是累积步数。若损失原本按 batch 取平均，应保证每个微批的梯度缩放与整批等价。

```

728
729     # Update parameters
730     optimizer.step()
731     optimizer.zero_grad(set_to_none=True)
732
733
734 def gradient_accumulation():
735     Large batch sizes: improve training stability
736     However, activation memory scales with batch size, so might run out.
737     B = 1024
738     L = 16
739     D = 1024
740     activation_memory = B * L * D
741     Gradient accumulation:
742     • Compute gradient on micro batches
743     • Accumulate the gradients (don't zero it out)
744     • Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients
745     micro_batch_size = 256
746     activation_memory = micro_batch_size * L * D
747

```

图 27：梯度累积在 micro-batch 上逐次反向，只在达到目标 batch 后更新并清零。²⁷

梯度累积不会减少整个大 batch 的主要 FLOPs，但会增加 kernel launch、数据循环与某些归一化差异；微批过小还会降低 GEMM 尺寸和 MFU。因此它解决的是容量约束，不保证吞吐不变。

11.2 激活检查点：前向少存，反向重算

普通训练为反向保存每层激活。Activation checkpointing（也称 gradient checkpointing 或 re-materialization）只保留部分边界激活；反向到某个区间时，从最近的检查点重新执行前向以恢复缺失中间量。

²⁷ 视频画面时间区间：01:13:00–01:13:15。

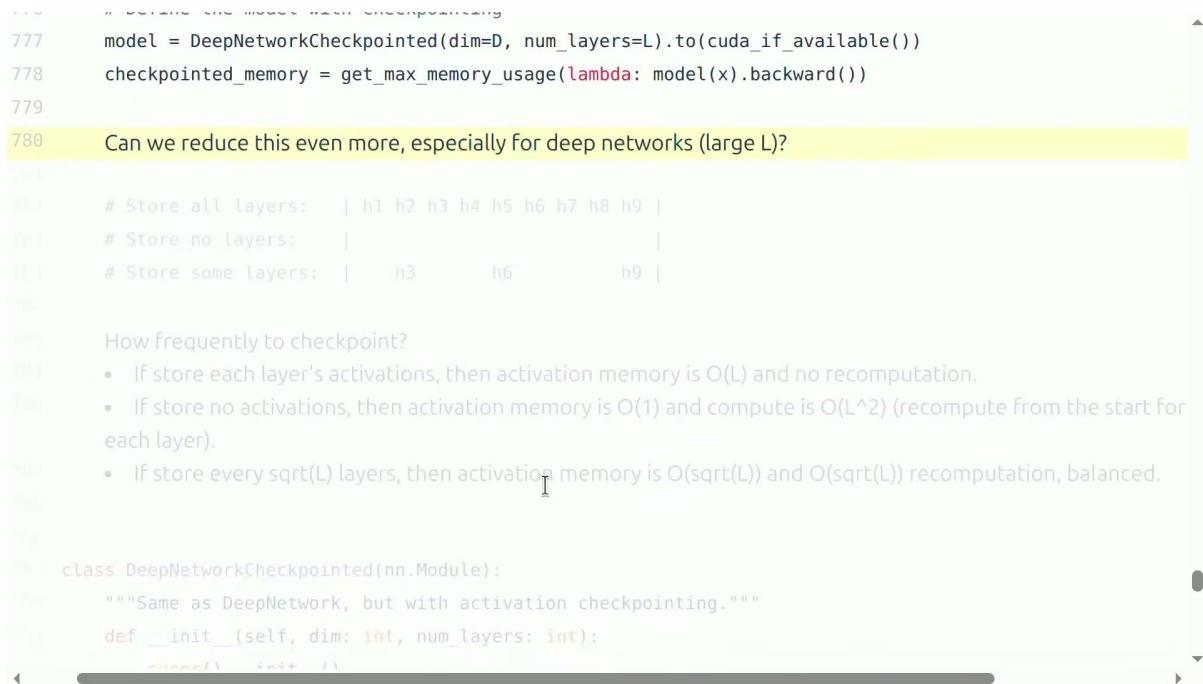


图 28: 检查点策略只保存部分层的激活, 并在反向阶段重算其余激活。²⁸

11.3 检查点频率的复杂度折中

把网络抽象成 L 层:

- 保存每层: 激活内存 $O(L)$, 无额外重算;
- 不保存中间层: 激活内存可到 $O(1)$, 但若每层都从头恢复, 计算可到 $O(L^2)$;
- 每隔约 $\sqrt{L}$ 层保存: 激活内存约 $O(\sqrt{L})$, 额外重算保持 $O(L)$ 量级。

²⁸ 视频画面时间区间: 01:14:30–01:14:45。

```

778 checkpointed_memory = get_max_memory_usage(lambda: model(x).backward())
779
780 Can we reduce this even more, especially for deep networks (large L)?
781
782 # Store all layers: | h1 h2 h3 h4 h5 h6 h7 h8 h9 |
783 # Store no layers: |                               |
784 # Store some layers: |   h3       h6       h9   |
785
786 How frequently to checkpoint?
787 • If store each layer's activations, then activation memory is  $O(L)$  and no recomputation.
788 • If store no activations, then activation memory is  $O(1)$  and compute is  $O(L^2)$  (recompute from the start for each layer).
789 • If store every  $\sqrt{L}$  layers, then activation memory is  $O(\sqrt{L})$  and  $O(\sqrt{L})$  recomputation, balanced.
790
791
792 class DeepNetworkCheckpointed(nn.Module):
793     """Same as DeepNetwork, but with activation checkpointing."""
794     def __init__(self, dim: int, num_layers: int):
795         super().__init__()
796         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])

```

图 29: 从全存到全不存, 检查点频率决定激活显存与重计算开销。²⁹

实践中还要选择检查点边界: 把昂贵、低内存的算子放进重算区可能不划算; 把内存大、重算便宜的区段检查点化通常收益更好。应测量峰值显存、每步时间和数值一致性, 而不是只看理论复杂度。

显存不足时的排查顺序

先测量峰值发生在哪个阶段; 确认是否存在意外 FP32、隐式复制或过大临时量; 再调 micro-batch 与梯度累积; 随后对高激活区启用检查点; 最后才扩大并行或采用更复杂分片。每一步同时记录吞吐和收敛行为。

11.4 本章小结

梯度累积把大 batch 的激活分时处理, 检查点把部分激活从“存储”改为“重算”。两者都用额外执行开销换容量, 因此必须与 MFU、数值等价性和端到端吞吐一起验收。

²⁹ 视频画面时间区间: 01:15:45–01:16:00。

12 总结与延伸

```

59     gradient_accumulation()
60     activation_checkpointing()
61
62     Summary:
63     • Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)
64     • einops: better way to think about tensor operations
65     • 6 (# data points) (# parameters) FLOPs per training step
66     • Arithmetic intensity / roofline analysis: compute-bound or memory-bound?
67     • Matrix multiplications are compute-bound, elementwise operations are memory-bound
68     • Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes
69
70
71 def motivating_questions():
72     Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 B100s?
73     total_flops = 6 * 70e9 * 15e12
74     h100_flop_per_sec = 1979e12 / 2
75     mfu = 0.5
76     flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
77     days = total_flops / flops_per_day
78

```

图 30: 本讲总结：张量统一表示训练状态，6BP 估算计算，Roofline 判断瓶颈，累积与检查点降低显存。³⁰

12.1 讲者总结

本讲把语言模型训练还原成一组可核算的张量操作：

1. 所有核心状态都落在具有形状、dtype、device 和生命周期的张量上；
2. 具名维度让运算语义、FLOPs 公式和代码实现相互对应；
3. 稠密短上下文训练可用 6PT 估算总计算；
4. 算术强度与 Roofline 判断算子是 compute-bound 还是 memory-bound；
5. 大矩阵乘法通常计算受限，逐元素与矩阵向量操作通常带宽受限；
6. 梯度累积和激活检查点以时间或重计算换取更低峰值显存。

12.2 结构化提炼：四张表贯穿后续课程

对任何新模型或 kernel，可以维护四张最小账表：

1. **张量表**：名字、语义形状、dtype、device、生命周期、字节数；
2. **计算表**：每个算子的 FLOPs、调用次数和总计算量；
3. **搬运表**：HBM、片上存储和互连之间的字节数与复用；
4. **时间表**：理论下界、实测时间、MFU、瓶颈与异常同步。

³⁰ 视频画面时间区间：01:17:00–01:17:15。

这四张表让架构、系统和训练优化使用同一语言：改变注意力形状时能看到 FLOPs 和激活变化，改变 dtype 时能看到容量、带宽和峰值变化，改变并行策略时能看到通信和驻留副本变化。

12.3 建议练习

1. 复算 70B/15T/1024 H100 的 144 天结论，并分别将 MFU 改为 0.35 和 0.6，比较墙钟敏感性。
2. 为一个含 embedding、attention、MLP 和 AdamW 的小 Transformer 建立完整张量表，区分十进制 GB 与二进制 GiB。
3. 分别推导点积、矩阵向量和矩阵乘法的算术强度，并在不同 batch 下画 Roofline 位置。
4. 用 PyTorch 写同步正确的 matmul 基准，比较 FP32、BF16 与不同形状的 MFU；同时记录峰值显存。
5. 验证梯度累积与整批训练的梯度是否等价，并比较不同 micro-batch 对吞吐的影响。
6. 对深层网络尝试不同检查点间隔，画出峰值显存与每步时间的 Pareto 曲线。

12.4 拓展阅读

- Stanford CS336 Spring 2026: <https://cs336.stanford.edu/>
- Lecture 2 可执行课件: [lecture_02.py](#)
- Einops 教程: <https://einops.rocks/1-einops-basics/>
- Mixed Precision Training: <https://arxiv.org/abs/1710.03740>
- JAX Scaling Book Roofline: <https://jax-ml.github.io/scaling-book/roofline/>
- Transformer 显存核算: <https://erees.dev/transformer-memory/>

12.5 本章小结

资源核算的最终目的不是得到一串静态数字，而是建立可迭代的因果模型：形状决定元素与 FLOPs，dtype 决定字节和峰值，复用决定算术强度，硬件屋顶决定可实现吞吐，生命周期决定峰值显存。后续所有架构与系统优化，都可以回到这套账本验证。